

Technical Analysis of a Robust Proximal Policy Optimization Implementation

for Gymnasium MountainCarContinuous-v0

A code-faithful analysis of architecture, mathematics, optimization, execution flow,
reproducibility, and implementation trade-offs

Contents

Abstract	3
1 Introduction	3
2 Project Overview	3
3 Technical Foundations	4
3.1 Markov decision process	4
3.2 Continuous Gaussian policy	5
3.3 Policy-gradient advantage estimation	5
3.4 PPO clipped objective	5
3.5 Observation normalization	6
4 Data and Input Processing	6
5 System Architecture and Code Organization	6
5.1 Configuration layer	7
5.2 Reproducibility utilities	7
5.3 Dependency gate	7
5.4 Neural-network initialization	7
5.5 Actor-critic model	7
5.6 GAE utility	7
5.7 Evaluation and diagnostics	8
5.8 Artifact management	8
6 Detailed Methodology	8
6.1 Environment initialization	8
6.2 Rollout collection	8
6.3 Bootstrap and target construction	8
6.4 Minibatch PPO updates	9
6.5 Progress reporting	9
6.6 Checkpointing	9
7 Mathematical Formulation	9
8 Optimization and Learning Procedure	11
9 Implementation Details	11
9.1 Network structure	11
9.2 Device handling	11
9.3 Numerical stability	11
9.4 Smoke test	11
10 Execution Workflow	12
11 Design Decisions and Rationale	12
12 Computational Considerations	13

13 Reproducibility and Reliability	13
14 Limitations and Technical Considerations	13
14.1 Bounded-action and log-probability mismatch	13
14.2 Exploration scale is unconstrained	14
14.3 Terminal-versus-truncation handling	14
14.4 Minibatch-local advantage normalization	14
14.5 Evaluation diagnostics are first-episode focused	14
14.6 Output management does not equal scientific validation	14
15 Overall Technical Assessment	15
16 Conclusion	15
A Configuration Reference	16
B Artifact Reference	16

Abstract

The Python program implements a compact reinforcement-learning system built around Proximal Policy Optimization (PPO) for the continuous-control environment `MountainCarContinuous-v0`. The implementation combines a Gaussian actor-critic model, generalized advantage estimation (GAE), clipped PPO updates, observation normalization, orthogonal neural-network initialization, gradient clipping, periodic checkpointing, deterministic evaluation, structured artifact export, and Google Colab-oriented packaging. The program is designed around a single hidden-size configuration, a fixed primary random seed, rollout collection in batches, minibatch optimization across multiple epochs, and a final evaluation phase over independent evaluation seeds.

This report examines the project at three levels: the reinforcement-learning problem, the mathematical algorithm implemented by the code, and the software architecture that realizes it. It gives particular attention to the difference between actions sampled from the policy and actions clipped before they reach the environment, since this affects how closely the implementation matches the standard PPO objective. The report does not infer performance from logging, plots, checkpoints, or summary fields. It also distinguishes behavior that is explicit in the code from conclusions inferred from it.

The analysis is based on the source code. No additional experiments or externally observed results are assumed.

1 Introduction

The project addresses a standard continuous-control reinforcement-learning problem: learning a policy that selects a bounded continuous action from the environment state. The environment identifier is fixed to `MountainCarContinuous-v0`, while most algorithmic and execution settings are centralized in a `Config` dataclass. The source describes the implementation as a robust PPO variant, emphasizing exploratory variance, generalized advantage estimation, and automated artifact export in Google Colab.

The implementation is self-contained. Rather than relying on a high-level reinforcement-learning framework, it defines the policy/value network, seed handling, device selection, observation normalization, GAE computation, evaluation, checkpointing, metrics export, and plotting directly in Python. As a result, the main parts of the algorithm are visible at the level of tensors, probability distributions, buffers, optimization steps, and files written to disk.

The purpose of this report is to describe what the program actually does, explain the relevant mathematics, identify its implementation choices, and state its limitations without adding unsupported evidence. A logging or artifact-generation facility is treated as a software feature, not as evidence that the resulting measurements are scientifically valid or that training achieved the intended objective.

2 Project Overview

The program follows this computational pipeline:

1. Load or require the Gymnasium dependency and select CPU or CUDA execution.
2. Set random seeds for Python, NumPy, and PyTorch.

3. Create the continuous-control environment and read its observation and action dimensions and bounds.
4. Normalize observations to an approximately symmetric interval.
5. Instantiate a separate actor and critic inside an `ActorCritic` module.
6. Collect a fixed-length rollout of observations, sampled actions, log-probabilities, rewards, termination flags, and value estimates.
7. Bootstrap the value of the final rollout state and compute GAE advantages and corresponding return targets.
8. Run several epochs of minibatch PPO optimization using the clipped surrogate objective, a squared value loss, an entropy bonus, and gradient clipping.
9. Periodically report recent return, recent success rate, policy standard deviation, actor loss, and critic loss.
10. Periodically serialize model, optimizer, episode count, and configuration state to a checkpoint.
11. After training, evaluate the learned policy deterministically over multiple independently seeded episodes.
12. Save episode metrics, evaluation metrics, diagnostic arrays, an optional evaluation video, a JSON summary, a JSON configuration file, and a training-return plot.
13. Optionally archive the output directory and trigger a browser download when running in Google Colab.

The result is an on-policy actor-critic training system contained in a single Python file. Environment interaction, learning, evaluation, and artifact management are kept logically separate even though they live in one module.

3 Technical Foundations

3.1 Markov decision process

The code follows the standard Markov decision process formulation. At each time step, the agent observes a state or observation s_t , samples an action a_t from a policy $\pi_\theta(a \mid s)$, receives a scalar reward r_t , and transitions to the next observation s_{t+1} . The value function represented by the critic is

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right], \quad (1)$$

where $\gamma \in [0, 1)$ is the discount factor. The implementation fixes $\gamma = 0.99$.

The actor and critic represent two different functions. The actor parameterizes the action distribution, while the critic approximates $V^\pi(s)$. Using the critic as a baseline reduces the variance of the policy-gradient estimate compared with using raw returns alone.

3.2 Continuous Gaussian policy

The actor predicts a mean vector and maintains a state-independent log standard deviation parameter. For observation s , the code forms

$$\mu_\theta(s) = f_\theta(s), \quad \log \sigma_\theta = \phi, \quad \sigma_\theta = \exp(\phi), \quad (2)$$

and constructs an independent Gaussian distribution for the action dimensions:

$$\pi_\theta(a \mid s) = \mathcal{N}\left(a; \mu_\theta(s), \text{diag}(\sigma_\theta^2)\right). \quad (3)$$

Because the environment has one continuous action dimension, the distribution is effectively a one-dimensional Normal whose mean depends on the state while its standard deviation is shared across states.

For a vector action, the log-probability is computed as the sum of the per-dimension Normal log-probabilities:

$$\log \pi_\theta(a \mid s) = \sum_{j=1}^{d_a} \log \mathcal{N}(a_j; \mu_j(s), \sigma_j). \quad (4)$$

The policy entropy is treated similarly as the sum of the dimension-wise entropies.

3.3 Policy-gradient advantage estimation

The policy is not optimized directly against complete episodic returns. Instead, the code first constructs a temporal-difference residual:

$$\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t), \quad (5)$$

where d_t is the code-level done flag used to suppress bootstrapping after a completed transition. GAE then accumulates these residuals recursively:

$$A_t^{\text{GAE}(\gamma, \lambda)} = \delta_t + \gamma\lambda(1 - d_t)A_{t+1}^{\text{GAE}(\gamma, \lambda)}. \quad (6)$$

The implementation uses $\lambda = 0.95$, and the return targets are formed as

$$\hat{R}_t = \hat{A}_t + V(s_t). \quad (7)$$

The same construction supplies a regression target for the critic and an advantage estimate for the actor.

3.4 PPO clipped objective

The policy update uses the probability ratio between the updated policy and the behavior policy that generated the rollout:

$$r_t(\theta) = \exp(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)). \quad (8)$$

The code writes the clipped surrogate as a loss to be minimized:

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\max \left(-\hat{A}_t r_t(\theta), -\hat{A}_t \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \right) \right], \quad (9)$$

with $\epsilon = 0.20$. The sign convention follows from minimizing the loss rather than maximizing the objective.

The critic loss is mean squared error with an explicit factor of $1/2$:

$$L_V = \frac{1}{2} \mathbb{E}_t \left[(V_\psi(s_t) - \hat{R}_t)^2 \right]. \quad (10)$$

Entropy is included as a regularizer. Let H_t denote the policy entropy for a sampled observation. The total loss is

$$L_{\text{total}} = L_{\text{actor}} + c_V L_V - c_H \mathbb{E}_t[H_t], \quad (11)$$

where $c_V = 0.5$ and $c_H = 0.005$.

3.5 Observation normalization

The code maps each observation coordinate from its environment bounds to $[-1, 1]$ using

$$\tilde{x} = \text{clip} \left(2 \frac{x - \ell}{h - \ell} - 1, -1, 1 \right), \quad (12)$$

where ℓ and h are the lower and upper observation bounds. This transformation is applied before observations enter the networks.

4 Data and Input Processing

The environment is created with `gym.make(cfg.env_id)` using the fixed identifier `MountainCarContinuous-v0`. The source reads `observation_space.low` and `observation_space.high` and obtains the observation and action dimensions from the corresponding shapes. For the smoke test, the code explicitly asserts an observation shape of $(2,)$ and an action shape of $(1,1)$ at the model-output level, which is consistent with the expected low-dimensional structure of this environment.

No dataset is loaded from disk. Training data are generated through online interaction with the environment. Each rollout is an on-policy sample from the current policy and is used in the subsequent PPO optimization phase. The rollout buffer stores six fields: normalized observations, raw sampled actions, old log-probabilities, rewards, done flags, and value predictions.

Observation normalization is deterministic given the observation bounds and does not require estimating empirical moments from the rollout. This keeps the preprocessing simple and avoids maintaining separate running statistics. Normalization is tied to the declared environment bounds rather than the empirical range observed during training.

Each completed episode is reset with a seed equal to the base seed plus the episode count, producing a deterministic sequence of integer seeds for a fixed software environment. Evaluation uses a separate seed offset of 100000.

5 System Architecture and Code Organization

The source can be viewed as eight functional layers.

5.1 Configuration layer

The `Config` dataclass centralizes environment, optimization, evaluation, checkpointing, output, device, and rendering parameters. This keeps hyperparameters in one place and allows the full configuration to be serialized at the end of training.

The main values are 1000 total episodes, 2048 rollout steps, $\gamma = 0.99$, GAE $\lambda = 0.95$, PPO clipping $\epsilon = 0.20$, an Adam learning rate of 3×10^{-4} , ten optimization epochs, a minibatch size of 64, a value coefficient of 0.5, an entropy coefficient of 0.005, a gradient-norm cap of 0.5, a hidden width of 64, and 20 evaluation episodes.

5.2 Reproducibility utilities

The `seed_everything` function seeds Python’s standard random generator, NumPy, PyTorch CPU randomness, and all CUDA devices when CUDA is available. The device selector then maps the configuration string to either CUDA or CPU. If CUDA is explicitly requested but unavailable, the function raises a runtime error rather than silently falling back.

The source does not enable all PyTorch deterministic-execution options. The seeding strategy therefore improves reproducibility but does not guarantee bitwise-identical results across hardware, drivers, or library builds.

5.3 Dependency gate

The `require_gym` helper performs a lazy import of Gymnasium and reports an installation command when the dependency is missing. This keeps the module importable until an environment is actually needed and improves the clarity of missing-dependency errors.

5.4 Neural-network initialization

The `layer_init` helper applies orthogonal initialization to linear-layer weights and constant initialization to biases. The final actor-mean layer uses a smaller standard-deviation parameter, 0.01, while the final critic layer uses 1.0. This creates a smaller initial policy mean scale while keeping the value output at a standard scale.

5.5 Actor-critic model

The `ActorCritic` class contains two independent multilayer perceptrons. The critic has two hidden layers with tanh activations followed by a scalar output. The actor mean network has the same two-hidden-layer topology followed by a vector output of dimension equal to the action dimension. The log standard deviation is a separate trainable parameter named `actor_logstd`; it is not state-dependent.

5.6 GAE utility

The `compute_gae` function is an explicit reverse-time implementation. It iterates from the end of the rollout toward the beginning, tracks the exponentially weighted residual accumulator, and returns both advantages and return targets. The function is independent of PyTorch autograd because the rollout-time values are treated as fixed targets during optimization.

5.7 Evaluation and diagnostics

The `evaluate` function switches the model to evaluation mode and uses deterministic mean actions. It records episode return, length, and a Boolean success indicator. For the first evaluation episode it additionally stores RGB frames, positions, velocities, and actions so that a video and a compact diagnostic array can be written after evaluation.

5.8 Artifact management

The training function creates subdirectories for CSV files, plots, videos, checkpoints, reports, metadata, and logs. After training, it writes the corresponding artifacts and can package the output directory as a ZIP archive. In Google Colab, the script can also trigger a browser download.

6 Detailed Methodology

6.1 Environment initialization

Training begins by loading the environment, seeding the random sources, selecting the device, and creating the output directories. Observation bounds are captured once from the environment and reused by the normalization function. The model and Adam optimizer are instantiated before the first environment reset.

6.2 Rollout collection

The outer loop runs until the configured number of episodes has been completed. Each iteration creates a fresh rollout buffer and collects up to 2048 environment transitions; the loop can end earlier when the target episode count is reached.

At every step, the current normalized state is converted to a one-element batch tensor. Under `torch.no_grad()`, the actor-critic produces a sampled action, its log-probability, and its value estimate. The action tensor is copied back to NumPy and clipped to the environment interval before being passed to `env.step`.

The rollout buffer stores the raw sampled action rather than the clipped action. This distinction is examined later in the report.

Episode accounting occurs simultaneously with rollout collection. A transition is treated as done when either `terminated` or `truncated` is true. A Boolean success state is set only when `terminated` is true, making the code-level definition of success stricter than simply reaching the end of an episode.

When an episode ends, the program appends the return, length, and success indicator to the logging structures and immediately resets the environment with a deterministic episode-specific seed. The success log therefore contains episode-level binary outcomes rather than per-step rewards or termination probabilities.

6.3 Bootstrap and target construction

After a rollout, the code evaluates the current value function at the current state and uses that scalar as the bootstrap value for the final transition. The collected rewards, values, done flags,

and this bootstrap estimate are passed to the GAE function. The resulting advantages and return targets are then converted to PyTorch tensors.

Rollout collection and gradient-based learning are separate. During collection, the model is in evaluation mode with gradients disabled. During optimization, it is switched to training mode and the stored rollout data are reused for ten epochs.

6.4 Minibatch PPO updates

For each epoch, the rollout indices are shuffled using NumPy. The program then walks through contiguous minibatches of size 64. For each minibatch it reconstructs new log-probabilities and values from the current network parameters while conditioning the log-probability calculation on the stored action values.

Advantages are normalized separately within each minibatch:

$$\tilde{A} = \frac{A - \text{mean}(A)}{\text{std}(A) + 10^{-8}}. \quad (13)$$

This differs from PPO implementations that normalize advantages once over the entire rollout. The source re-standardizes the advantages for each minibatch, so it does not use a single global rollout mean and standard deviation.

The actor loss takes the maximum of the unclipped and clipped negative-surrogate terms, which is the minimization form of the PPO clipped objective. The critic uses the squared difference between predicted values and GAE return targets. The total loss combines the actor loss, half-weighted critic loss, and a negative entropy term.

The optimizer is Adam with learning rate 3×10^{-4} and $\epsilon = 10^{-5}$. Gradients are clipped to a norm of at most 0.5 before each optimizer step. Loss values are kept in Python lists for progress reporting.

6.5 Progress reporting

Every 20 completed episodes, once at least 20 episodes have been logged, the program reports recent mean return, recent success rate, current mean policy standard deviation, average actor loss, and average critic loss over the minibatches processed in the latest update. These values are diagnostics, not a formal statistical evaluation.

6.6 Checkpointing

Every 200 completed episodes, the code writes a PyTorch checkpoint containing the current episode count, model state dictionary, optimizer state dictionary, and a dictionary form of the configuration. The checkpoint filename is fixed to `agent_latest.pt`, so each new checkpoint replaces the previous one rather than creating a numbered sequence.

7 Mathematical Formulation

This section collects the core equations using a consistent notation and relates them directly to the implementation.

Let s_t denote the normalized observation at time t , let a_t denote the stored policy action, and let r_t denote the reward returned by the environment. The actor produces a mean vector $\mu_\theta(s_t)$ and a state-independent standard deviation $\sigma = \exp(\phi)$. The Gaussian policy is

$$\pi_\theta(a_t | s_t) = \mathcal{N}(a_t; \mu_\theta(s_t), \text{diag}(\sigma^2)). \quad (14)$$

The code’s value estimate is a scalar

$$V_\psi(s_t) = f_\psi(s_t), \quad (15)$$

where ψ denotes critic parameters. The temporal-difference residual is

$$\delta_t = r_t + \gamma(1 - d_t)V_\psi(s_{t+1}) - V_\psi(s_t). \quad (16)$$

The GAE recursion is

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}. \quad (17)$$

The return target is

$$\hat{R}_t = \hat{A}_t + V_\psi(s_t). \quad (18)$$

The PPO probability ratio is

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (19)$$

The ratio is computed in log space before exponentiation:

$$r_t(\theta) = \exp(\log \pi_\theta(a_t | s_t) - \log \pi_{\theta_{\text{old}}}(a_t | s_t)). \quad (20)$$

With clipping parameter ϵ , the policy loss minimized by the program is

$$L_\pi = \mathbb{E}_t \left[\max \left(-\hat{A}_t r_t(\theta), -\hat{A}_t \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \right) \right]. \quad (21)$$

The critic loss is

$$L_V = \frac{1}{2} \mathbb{E}_t \left[(V_\psi(s_t) - \hat{R}_t)^2 \right]. \quad (22)$$

Finally,

$$L = L_\pi + c_V L_V - c_H \mathbb{E}_t[H_\theta(s_t)]. \quad (23)$$

The action preprocessing performed after sampling is distinct from the Gaussian model itself. The sampled action a_t^{raw} is transformed to

$$a_t^{\text{env}} = \text{clip}(a_t^{\text{raw}}, -1, 1), \quad (24)$$

and a_t^{env} is passed to the environment, while a_t^{raw} remains in the rollout buffer for later log-probability evaluation. This differs from a policy whose density is defined directly over bounded actions or over a formally transformed distribution, and it affects the exact policy-gradient interpretation.

8 Optimization and Learning Procedure

The optimization problem alternates between data collection and parameter updates. The rollout policy is fixed during collection because the network is used under no-gradient inference. The collected batch is then reused for several optimization epochs, which is a defining feature of PPO.

The learning schedule has a fixed rollout length rather than a fixed number of updates per episode. Because episode boundaries can occur inside the rollout, a single update can contain transitions from several episodes. The GAE routine uses the done flags to avoid crossing episode boundaries when propagating advantage estimates.

The actor and critic use the same Adam learning rate because both are passed to a single optimizer. There is no separate critic learning rate, learning-rate scheduler, value-loss clipping, or adaptive KL penalty in the implementation.

Exploration is controlled by `actor_logstd`. It starts from zero, corresponding to $\sigma = 1$. Because the parameter is trainable and state-independent, the policy learns one global scale per action dimension rather than an observation-dependent covariance. The entropy coefficient provides an additional pressure against complete variance collapse, although the source does not explicitly constrain the log standard deviation to a minimum or maximum value.

9 Implementation Details

9.1 Network structure

For the two-dimensional observation and one-dimensional action space, the actor and critic are small multilayer perceptrons. Each uses two hidden layers of width 64 with tanh activations. The model is small and suited to a low-dimensional control task. It is not intended as a general-purpose representation learner but as a compact policy/value approximator.

9.2 Device handling

All training tensors are placed on the device selected by `choose_device`. Environment interaction itself remains CPU-side because Gymnasium returns NumPy arrays. The code converts environment outputs to tensors for the model calls and converts actions back to NumPy before stepping the environment.

9.3 Numerical stability

The code includes several numerical safeguards. Observation values are clipped during normalization. Advantage normalization includes an $\epsilon = 10^{-8}$ denominator offset. Gradient norms are clipped. The smoke test asserts that sampled actions, log-probabilities, and value predictions are finite. These choices reduce the risk of silent numerical failures but do not guarantee numerical robustness.

9.4 Smoke test

The `smoke_test` function is small by design. It creates the target environment, resets it, verifies the observation shape, instantiates a matching actor-critic model, performs a single forward pass, and checks output shapes and finiteness. It is an integration sanity check rather than a test of

training validity. It does not verify GAE, the PPO loss, checkpoint restoration, file export, or convergence.

10 Execution Workflow

When the file is executed as a script, Python enters the `main` guard. The smoke test runs first. The configuration is then instantiated with 1000 episodes, the default output directory, rendering enabled, and automatic Colab download enabled. The `train` function is invoked, after which the final summary dictionary is printed as formatted JSON.

Within `train`, output directories are created before the environment and model are initialized. Training continues until the configured number of completed episodes is reached. The environment is closed before final metrics are written. Evaluation then occurs in a separate environment instance. The final artifacts are generated after this training and evaluation sequence.

The program state consists of several interacting pieces: Python random generators, NumPy random state, PyTorch model parameters, optimizer state, environment state, episode counters, rollout buffers, and filesystem outputs. The checkpoint captures only part of this state. It stores the model and optimizer states and the episode count, but not all random-number-generator states or the active environment state.

11 Design Decisions and Rationale

Several choices are apparent from the source structure and comments.

First, the implementation places the primary hyperparameters in a dataclass, which makes experimentation and serialization easier than scattering them across functions.

Second, the policy mean output uses a much smaller orthogonal initialization scale than the critic output. This starts the mean action near a relatively small magnitude while leaving exploration controlled by the explicitly parameterized standard deviation.

Third, the entropy coefficient is nonzero. Together with an initial log standard deviation of zero, this makes exploration an explicit design feature. The source description also frames the design as a way to avoid a local-optimum trap through high exploratory variance.

Fourth, evaluation is deterministic. The evaluation function uses the policy mean instead of sampling from the Normal distribution. This gives a direct measure of the learned mean policy, but it does not characterize stochastic-policy performance.

Fifth, the output hierarchy separates episode metrics, evaluation metrics, checkpoints, reports, metadata, logs, plots, and videos. This makes the run easier to inspect without relying on the console log as the only record.

These are mainly implementation-level inferences. Some are stated explicitly in the source comments; others are inferred from the code structure. The report does not attribute motivations that are not supported by the source.

12 Computational Considerations

The main computational cost of each training update comes from repeated forward and backward passes through the actor and critic over the rollout. For fixed network architecture and rollout size B , the dominant neural computation grows approximately linearly with the number of stored transitions. The implementation performs $E = 10$ optimization epochs, so the same rollout is traversed multiple times.

The target environment has very small observation and action spaces, so model parameters use little memory compared with rollout storage and output artifacts. The rollout buffer contains Python lists that are converted into NumPy arrays and then tensors before optimization. This is simple and transparent, but less memory-efficient than preallocated contiguous buffers at larger scales.

The program also records one evaluation episode as raw RGB frames and then encodes them to MP4. Video generation can therefore be more I/O-sensitive than the learning loop. The training-return plot is similarly a post-processing cost and is not part of the gradient computation.

The fixed 2048-step rollout can span multiple MountainCarContinuous episodes. This is expected for on-policy PPO and keeps the batch size independent of the exact episode termination times.

13 Reproducibility and Reliability

The implementation includes several reproducibility mechanisms: a central configuration, deterministic seed assignment by episode index, serialization of the configuration to JSON, serialization of model and optimizer states to a checkpoint, and a fixed evaluation seed offset. These features make a run easier to audit.

Reproducibility is not absolute. The code seeds major random-number sources but does not force every CUDA and backend operation into deterministic modes. It also depends on the installed versions of Gymnasium, PyTorch, NumPy, imageio, FFmpeg, and related runtime components. The source does not record package versions in its metadata file. The saved configuration therefore describes the hyperparameters but not the complete software environment required for byte-level reproduction.

The checkpointing scheme also captures only the latest model and optimizer state. A resumed run would not automatically reconstruct the exact rollout stream or all random states used before the checkpoint. The checkpoint is useful for model recovery but is not a complete deterministic replay snapshot.

14 Limitations and Technical Considerations

14.1 Bounded-action and log-probability mismatch

The main technical issue is the relationship between Gaussian sampling, clipping, and PPO log-probabilities. The actor defines an unbounded Normal distribution and samples a raw action, but the environment receives the clipped action in $[-1, 1]$. PPO later evaluates the stored raw action under the updated Normal distribution. Thus, the optimized probability model describes the raw action, while the environment transition is generated by the clipped action.

For a policy-gradient method, the probability model should correspond to the action used to

generate the trajectory, or the policy density should account for the transformation between them. The current code does neither: clipping is applied externally, while the raw action remains the likelihood variable. This matters whenever samples fall outside the legal interval, because multiple raw Gaussian samples map to the same clipped environment action.

The source comments refer to “standard clipped Gaussian policy actions,” but the implementation is more precisely a Gaussian policy followed by deterministic clipping before the environment step. That distinction should be stated explicitly in a scientific description of the algorithm.

14.2 Exploration scale is unconstrained

The trainable `actor_logstd` starts at zero and is not explicitly bounded. The exponential mapping guarantees a positive standard deviation but does not prevent it from becoming very large or very small. The entropy term provides a soft pressure, not a hard constraint. Whether this is desirable depends on the learning dynamics; the code does not impose an explicit variance floor or ceiling.

14.3 Terminal-versus-truncation handling

The rollout buffer stores a combined `done` flag equal to `terminated` or `truncated`. The GAE recursion uses that combined flag to suppress bootstrapping. In environments where truncation represents a time-limit condition rather than a true terminal state, this choice can discard value information at the truncation boundary. The practical effect depends on the target environment and configured episode limits, but the code-level behavior is clear.

14.4 Minibatch-local advantage normalization

The program normalizes advantages independently inside each minibatch rather than once over the whole collected rollout. This changes the exact gradient estimator compared with implementations that use one global mean and standard deviation. It is an implementation choice that should be documented for reproducibility.

14.5 Evaluation diagnostics are first-episode focused

The evaluation function computes returns, lengths, and success flags across all evaluation episodes, but positions, velocities, actions, and frames are recorded only for the first evaluation episode. The final summary reports the maximum position from this first diagnostic sequence rather than an aggregate across evaluation episodes. That field is a single-trajectory diagnostic rather than a population statistic.

14.6 Output management does not equal scientific validation

The program creates many artifacts, including CSV files, JSON summaries, a plot, a video, and a checkpoint. These artifacts aid inspection, but their presence does not establish that the underlying result is correct, statistically stable, or independently replicated. The source does not include a repeated-seed study, confidence intervals, hypothesis tests, or cross-run variance analysis.

15 Overall Technical Assessment

The implementation is coherent as a compact PPO training system for a low-dimensional continuous-control environment. It includes the essential algorithmic pieces expected in a hand-written PPO implementation: a stochastic actor, a value baseline, rollout collection, GAE, the clipped surrogate objective, entropy regularization, minibatch optimization, gradient clipping, evaluation, and checkpointing. The configuration and artifact-management layers also make the program easier to inspect and reproduce than a minimal training script.

The source exposes the mechanics of PPO rather than hiding them behind a framework. The actor distribution, value estimation, advantage calculation, loss construction, and optimizer steps can all be traced directly through the code. The smoke test provides a basic guard against obvious shape and numerical failures before full training begins.

At the same time, the implementation should not be described as an exact bounded-action PPO formulation without qualification. The raw-versus-clipped action distinction is the main technical caveat because it affects the interpretation of the log-probability ratio. The treatment of truncation as terminal, the unconstrained policy standard deviation, and minibatch-local advantage normalization are additional implementation choices that may affect learning dynamics or comparability with other PPO implementations.

From a software-engineering perspective, the project uses centralized configuration, explicit device selection, seed setup, modular helper functions, checkpoint serialization, structured outputs, and a clear main execution path. From a scientific-computing perspective, its documentation is most reliable when it describes what the code actually computes rather than treating implementation comments as theoretical guarantees. A technical description should distinguish the intended PPO design from the exact behavior of the action pipeline and termination logic.

16 Conclusion

The program is a self-contained PPO implementation for a continuous-control environment, organized around a Gaussian actor-critic architecture and a conventional on-policy learning loop. Its main computational stages are easy to reconstruct: bounded observations are normalized, a Gaussian policy samples actions, rollout transitions are accumulated, GAE constructs advantages and return targets, PPO optimizes the actor and critic over minibatches, and the resulting policy is evaluated deterministically before structured artifacts are exported.

The project includes several useful elements of research-oriented software development: explicit configuration, modularity, numerical safeguards, reproducibility controls, checkpointing, evaluation, and artifact management. Its mathematical core is recognizable and largely faithful to standard PPO mechanics. The main qualification is that the environment action is clipped after sampling while PPO continues to evaluate the raw Gaussian action. This is a substantive implementation detail and should be disclosed whenever the method is described academically.

The code is a compact example of implementing a reinforcement-learning algorithm from first principles. Its value comes from the training loop as well as the visibility of the mathematical and computational decisions behind it. A sound presentation should highlight this separation of components and state the exact implementation choices and their technical consequences.

A Configuration Reference

Parameter	Value	Role
env_id	MountainCarContinuous-v0	Target Gymnasium environment.
seed	42	Primary training seed and base for episode-specific resets.
total_episodes	1000	Target number of completed training episodes.
rollout_steps	2048	Maximum transitions collected per PPO update.
max_steps_per_episode	999	Upper bound used during deterministic evaluation.
γ	0.99	Discount factor.
λ	0.95	GAE trace parameter.
clip_ratio	0.20	PPO probability-ratio clipping range.
learning_rate	3×10^{-4}	Adam step size.
train_epochs	10	Number of passes over each rollout.
minibatch_size	64	Optimization minibatch size.
value_coef	0.5	Weight on critic loss.
entropy_coef	0.005	Weight on entropy regularization.
max_grad_norm	0.5	Gradient-norm clipping threshold.
hidden_size	64	Width of each hidden layer.
eval_episodes	20	Number of deterministic evaluation episodes.
eval_seed_offset	100000	Numeric separation between training and evaluation seed sequences.
checkpoint_every	200	Training-episode interval for latest-checkpoint serialization.
print_every	20	Training-episode interval for progress reporting.
render_evaluation	True	Enable RGB rendering for the first evaluation episode.
auto_download_colab	True	Trigger ZIP packaging and browser download in Colab.

B Artifact Reference

The training function creates an output hierarchy containing the following artifacts.

Location	Purpose	Created by
csv/episode_metrics.csv	Per-episode return, length, and success indicator for training.	Training loop
csv/evaluation_metrics.csv	Return, length, and success for each evaluation episode.	Evaluation

Location	Purpose	Created by
logs/evaluation_diagnostics.npz	First evaluation trajectory diagnostics.	Evaluation
videos/evaluation_rollout.mp4	Optional first-episode rendered evaluation video.	Evaluation
checkpoints/agent_latest.pt	Latest model, optimizer, episode, and configuration state.	Checkpointing
reports/final_results.json	End-of-run summary statistics and elapsed time.	Finalization
metadata/config.json	Serialized configuration.	Finalization
plots/training_return.png	Training return curve and optional 50-episode moving average.	Plotting